

Name(1): Susi Sorglos

Abgabetermin:

Name(2): Max Mustermann

Punkte:

Übungsgruppe: Gruppe 1

korrigiert:

Beispiel 1 (24 Punkte) SimpleAnimal: Entwerfen Sie aus der nachfolgenden Spezifikation ein Klassendiagramm, instanzieren Sie dieses und implementieren Sie die Funktionalität entsprechend:

...bla bla bla...

Allgemeine Hinweise:

Verfassen Sie weiters eine Systemdokumentation entsprechend der Vorlage auf der Kommunikationsplattform. Verwenden Sie Doxygen-Kommentare für die Dokumentation des Komponentenentwurfes.

Führen Sie gemeinsam vor der Realisierung eine Aufwandsschätzung in (Ph) durch und notieren Sie die geschätzte Zeitdauer in der Systemdokumentation.

Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!**

Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren!

Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

Diese Hinweise gelten auch für alle nachfolgenden Übungen.



HSD

FH-HAGENBERG

Systemdokumentation Projekt SimpleAnimal

Version 1.0

M. Mustermann, S. Sorglos

Hagenberg, 6. Oktober 2025

Inhaltsverzeichnis

1	Organisatorisches	3
1.1	Team	3
1.2	Aufteilung der Verantwortlichkeitsbereiche	3
1.3	Aufwand	4
2	Anforderungsdefinition (Systemspezifikation)	5
3	Systementwurf	6
3.1	Klassendiagramm	6
3.2	Designentscheidungen	7
4	Dokumentation der Komponenten (Klassen)	7
5	Testprotokollierung	8
6	Quellcode	9
6.1	Object.h	9
6.2	Object.cpp	10
6.3	Animal.h	10
6.4	Animal.cpp	12
6.5	Cat.h	12
6.6	Cat.cpp	13
6.7	Dog.h	14
6.8	Dog.cpp	15
6.9	Zoo.h	15
6.10	Zoo.cpp	17
6.11	Testtreiber - SimpleAnimal.cpp	18

1 Organisatorisches

1.1 Team

- Max Mustermann, Matr.-Nr.: xxxx, E-Mail: Max.Mustermann@fh-hagenberg.at
- Susi Sorglos, Matr.-Nr.: yyyy, E-Mail: Susi.Sorglos@fh-hagenberg.at

1.2 Aufteilung der Verantwortlichkeitsbereiche

- Max Mustermann
 - Design Klassendiagramm
 - Implementierung und Test der Klassen:
 - * Object,
 - * Animal,
 - * Cat und
 - * Dog.
 - Dokumentation
- Susi Sorglos
 - Design Klassendiagramm
 - Implementierung und Komponententest der Klassen:
 - * Object und
 - * Zoo.
 - Implementierung des Testtreibers und Integrationstest des Gesamtsystems

– Dokumentation

1.3 Aufwand

- Max Mustermann: geschätzt 6 Ph / tatsächlich 9 Ph
- Susi Sorglos: geschätzt 6 Ph / tatsächlich 7 Ph

2 Anforderungsdefinition (Systemspezifikation)

In diesem System werden die Tiere eines Zoo's abgebildet und dort gespeichert. Die Tiere speichern das Gewicht in Kilogramm als Ganzzahl und werden mit einer forlaufenden Nummer identifiziert. Sie besitzen eine gemeinsame Schnittstelle die folgende Funktionalität liefert:

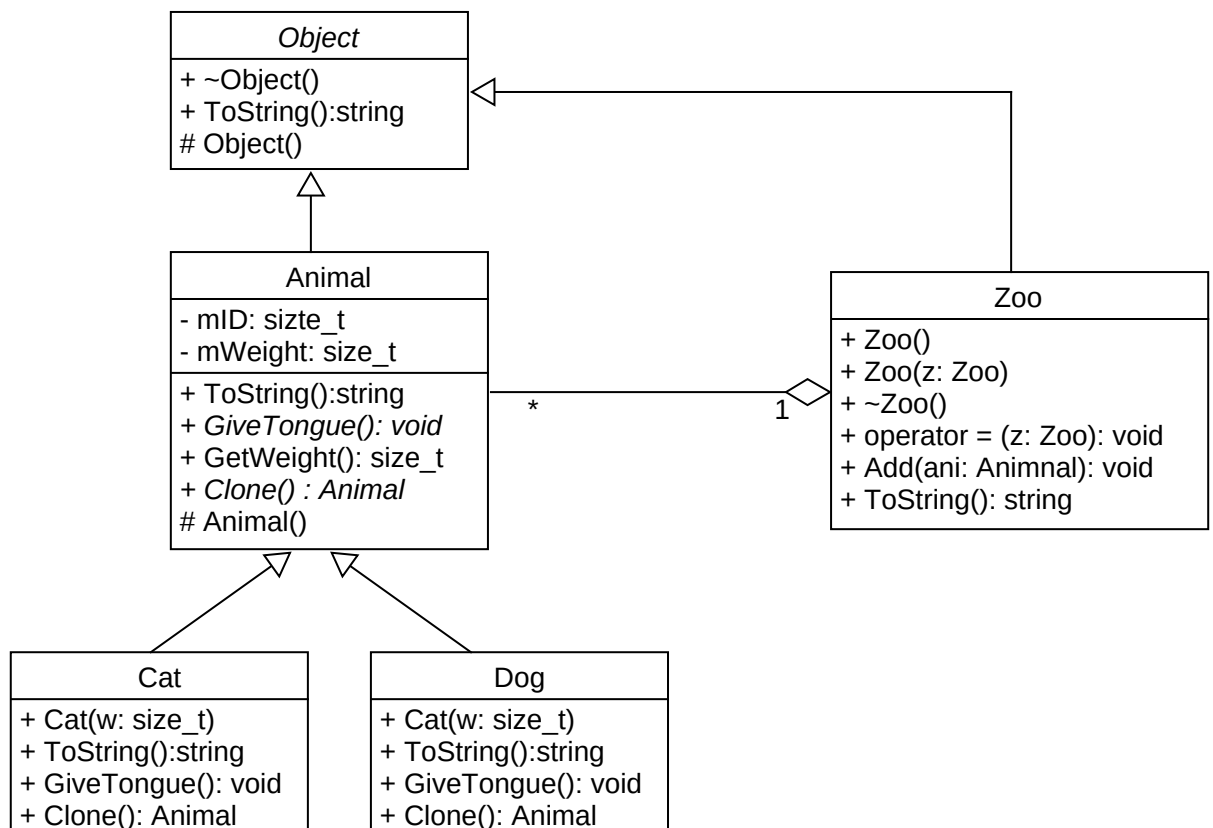
- Gib einen Laut (entsprechende Ausgabe auf `std::cout`).
- Liefere einen String mit den gespeicherten Attributen.
- Erstelle einen Klon von sich selbst.

Der Zoo speichert alle Tiere und besitzt die Tier-Objekte nach dem Hinzufügen. Via Zugriffsmethoden kann auf die Tiere zugegriffen werden und eine String-Methode liefert eine Repräsentation aller Tiere im Zoo mit allen Attributen als eine abgeschlossene Zeichenkette. Zusätzlich kann der Zoo inklusive all seiner Tiere kopiert und zugewiesen werden.

3 Systementwurf

3.1 Klassendiagramm

Hier wird das Klassendiagramm eingefügt. Sollte dieses nicht auf eine A4-Seite passen, so kann es in eine eingene pdf-Datei ausgelagert werden. Verweisen Sie an dieser Stelle auf diese Datei.



3.2 Designentscheidungen

Im Klassendiagramm wurde der Polymorphismus angewendet, um unterschiedliche Tierarten mit der gemeinsamen Schnittstelle 'Animal' anzusprechen. Die Klasse 'Zoo' speichert einen Container mit der abstrakte Basisklasse 'Animal' als Elementtyp und kann somit alle bestehenden und auch neuen Tierarten verwalten, die sich von der gemeinsamen Basisklasse 'Animal' ableiten.

Designentscheidungen sind von entscheidender Bedeutung für die Qualität und den Erfolg einer Softwareanwendung. Sie beeinflussen nicht nur die technische Umsetzung, sondern auch die Fähigkeit der Anwendung, zukünftigen Anforderungen gerecht zu werden und Änderungen effizient zu bewältigen. Sie beantworten meist folgende Fragen:

- Warum wurde die Klassenhierarchie so gewählt?
- Wurden Design Pattern verwendet und warum?
- Wurde Abstraktion und der Polymorphismus angewendet?
- Wie kann die Klassenstruktur einfach erweitert werden?
-

4 Dokumentation der Komponenten (Klassen)

Die Dokumentation der einzelnen Klassen und Komponenten erfolgt direkt im Quellcode mit Doxygen-Kommentaren. Erzeugen Sie danach eine HTML-Doku und verweisen Sie auf die Start-HTML-Datei.

Die HTML-Startdatei befindet sich im Verzeichnis [../doxy/html/index.html](#)

5 Testprotokollierung

```
Visual Leak Detector read settings from: C:\Program Files (x86)\Visu
Visual Leak Detector Version 2.5.1 installed.
Testing Invalid Animal Creation...
[PASS] Exception caught: Weight must be a positive number
[FAIL] Expected exception for invalid cat weight not thrown.
Testing Null Animal Addition to Zoo...
[PASS] Exception caught: null_pointer param in Zoo::Add(...)
Testing Animal Weights...
[PASS] Animal weight test passed.
[PASS] Animal weight test passed.
[PASS] Animal weight test passed.
Testing Animal IDs...
[PASS] Animal ID test passed.
[PASS] Animal ID test passed.
[PASS] Animal ID test passed.
Testing Animal Sounds...
[PASS] Animal sound test passed.
[PASS] Animal sound test passed.
[PASS] Animal sound test passed.
Testing Animal ToString...
[PASS] Animal ToString test passed.
[PASS] Animal ToString test passed.
[PASS] Animal ToString test passed.
Testing Animal Cloning...
[PASS] Animal cloning test passed.
[PASS] Animal cloning test passed.
Testing Zoo Contents...
[PASS] Zoo ToString test passed.
No memory leaks detected.
Visual Leak Detector is now exiting.
```

6 Quellcode

Die Klassen werden entsprechend der Klassenhierarchie von oben nach unten angegeben. Zuerst die Header-Datei, gefolgt von der Implementierung. Der Testtreiber (oder Client-Klassen) werden am Ende angegeben.

6.1 Object.h

```
1 #ifndef OBJECT_H
2 #define OBJECT_H
3 #include <string>
4
5 /// @author Franz Wiesinger
6 /// @date 10.06.2022
7 /// @brief Base class of all base classes.
8 ///
9 /// Is the common root class of all classes. If there is no other base class,
10 /// this class is to be selected as the base class.
11 class Object
12 {
13 public:
14     /// @brief Virtual default destructor -> once virtual, always virtual.
15     ///
16     /// It is not necessary to declare another virtual destructor
17     /// in derived classes.
18     virtual ~Object() = default;
19
20     /// @brief String representation of class Object.
21     /// @returns a string for class Object.
22     ///
23     /// Constructs a string representation of class Object
24     /// and can be overridden in derived classes.
25     virtual std::string ToString() const;
26
27 protected:
28     /// @brief Protected default constructor that makes the class abstract.
29     Object() = default;
30 };
31
32 #endif // OBJECT_H
```

6.2 Object.cpp

```
1 #include "Object.h"
2 #include <sstream>
3 #include <iostream>
4 #include "Object.h"
5
6 using namespace std;
7
8 std::string Object::ToString() const
9 {
10     {
11         ostringstream strStream;
12         strStream << "object_of_class_'Object:'_"
13                 << hex << this << endl;
14         return strStream.str();
15     }
16 }
```

6.3 Animal.h

```
1 #ifndef ANIMAL_H
2 #define ANIMAL_H
3 #include "Object.h"
4
5 /// @author Franz Wiesinger
6 /// @date 10.06.2022
7 /// @brief Abstract base class for all animals.
8 ///
9 /// The base class specifies the weight and a unique identifier
10 /// for each animal.
11 class Animal : public Object
12 {
13 public:
14     /// @brief Access function for the weight.
15     /// @returns the weight as a positive integer.
16     size_t GetWeight() const;
17
18     /// @brief Access function for the unique identifier.
19     /// @returns the identifier as a positive integer.
20     size_t GetId() const;
```

```
21
22     /// @brief Outputs the sound of the respective animal on std::cout.
23     ///
24     /// The method is pure virtual and must be implemented in
25     /// derived classes. Since the implementation is missing,
26     /// no object of this class can be created.
27     /// The class is therefore abstract.
28     virtual void GiveTongue() const = 0;
29
30
31     /// @brief String representation of class Animal.
32     ///
33     /// Constructs a string representation of class Animal
34     /// and can be overridden in derived classes.
35     /// @returns a string for class Animal.
36     std::string ToString() const override;
37
38     /// @brief Creates a clone of the animal.
39     ///
40     /// The method is pure virtual and must be implemented
41     /// in derived classes.
42     virtual Animal const* Clone() const = 0;
43
44 protected:
45     /// @brief Protected constructor of an animal.
46     /// @param[in] w is the weight of an Animal
47     Animal(size_t const w);
48
49 private:
50     /// @brief Generates a sequential number.
51     /// @details The generated number is used for the unique identifier.
52     static size_t msCounter;
53
54     /// @brief Unique identifier of an animal.
55     size_t mId;
56
57     /// @brief The weight of an Animal in kilogram.
58     size_t mWeight;
59 };
60
61
62
63 #endif //ANIMAL_H
```

6.4 Animal.cpp

```
1 #include <sstream>
2 #include "Animal.h"
3 using namespace std;
4
5
6 size_t Animal::msCounter = 0;
7
8 size_t Animal::GetWeight() const
9 {
10     return mWeight;
11 }
12
13 size_t Animal::GetId() const
14 {
15     return mId;
16 }
17
18 std::string Animal::ToString() const
19 {
20     ostringstream os;
21     os << "my_weight_is_" << mWeight << endl;
22     return os.str();
23 }
24
25 Animal::Animal(size_t const w) : mId{ msCounter++ }
26 {
27     if (w <= 0) {
28         throw std::invalid_argument("Weight_must_be_a_positive_number");
29     }
30     mWeight = w;
31 }
```

6.5 Cat.h

```
1 #ifndef CAT_H
2 #define CAT_H
3 #include "Animal.h"
4
5 /// @author Franz Wiesinger
```

```
6  /// @date 10.06.2022
7  /// @brief Concrete class for a cat.
8  ///
9  /// The class represents a cat and all of its attributes.
10 class Cat : public Animal
11 {
12 public:
13     /// @brief Constructor of a cat.
14     ///
15     /// Calls the constructor of an animal.
16     /// @param[in] w is the weight as positive integer.</param>
17     Cat(size_t const w);
18
19     /// @brief Outputs the sound of a cat.
20     ///
21     /// Implements the pure virtual method of class Animal.
22     void GiveTongue() const override;
23
24     /// @brief Returns a string representation of a cat.
25     /// @returns a string of a cat with all attributes.
26     std::string ToString() const override;
27
28     /// @brief Clones a cat and returns itself as clone-object.
29     /// @returns a cloned object of a cat.
30     Animal const* Clone() const override;
31 };
32 #endif //CAT_H
```

6.6 Cat.cpp

```
1  #include <iostream>
2  #include <sstream>
3  #include "Cat.h"
4  using namespace std;
5
6  Cat::Cat(size_t const w) : Animal{ w }
7  {}
8
9  void Cat::GiveTongue() const
10 {
11     cout << "miaow" << endl;
```

```
12 }
13
14 std::string Cat::ToString() const
15 {
16     ostringstream os;
17     os << "ID:" << GetId() << "_I'm_a_cat_and_" << Animal::ToString();
18     return os.str();
19 }
20
21 Animal const* Cat::Clone() const
22 {
23     return new Cat(*this);
24 }
```

6.7 Dog.h

```
1 #ifndef DOG_H
2 #define DOG_H
3 #include "Animal.h"
4
5 /// @author Franz Wiesinger
6 /// @date 10.06.2022
7 /// @brief Concrete class for a dog.
8 ///
9 /// The class represents a dog and all of its attributes.
10 class Dog : public Animal
11 {
12 public:
13     /// @brief Constructor of a dog.
14     /// @param[in] w is the weight as positive integer.
15     ///
16     /// Calls the constructor of an animal.
17     Dog(size_t const w);
18
19     /// @brief Outputs the sound of a dog.
20     ///
21     /// Implements the pure virtual method of class Animal.
22     void GiveTongue() const override;
23
24     /// @brief Returns a string representation of a dog.
25     /// @returns a string of a dog with all attributes.
```

```
26     std::string ToString() const override;
27
28     /// @brief Clones a dog and returns itself as clone-object.
29     /// @returns a cloned object of a dog.
30     Animal const* Clone() const override;
31 };
32 #endif //DOG_H
```

6.8 Dog.cpp

```
1 #include <iostream>
2 #include <sstream>
3 #include "Dog.h"
4 using namespace std;
5
6 Dog::Dog(size_t const w) : Animal{ w }
7 {}
8
9 void Dog::GiveTongue() const
10 {
11     cout << "bark" << endl;
12 }
13
14 std::string Dog::ToString() const
15 {
16     ostringstream os;
17     os << "ID:" << GetId() << "_I'm_a_dog_and_" << Animal::ToString();
18     return os.str();
19 }
20
21 Animal const* Dog::Clone() const
22 {
23     return new Dog(*this);
24 }
```

6.9 Zoo.h

```
1 #ifndef ZOO_H
2 #define ZOO_H
```

```
3 #include <vector>
4 #include "Animal.h"
5
6 /// @author Franz Wiesinger
7 /// @date 10.06.2022
8 /// @brief Concret class Zoo -> stores all animals in a polymorphic container.
9 ///
10 /// The class implements the copy constructor and the assignment operator and
11 /// gives access to the animals via iterators.
12 class Zoo : public Object
13 {
14 public:
15
16     /// @brief Default CTor: must be implemented first when the CopyCTor
17     /// is defined.
18     ///
19     /// The CopyCTor overrides the default constructor
20     Zoo() = default;
21
22     /// @brief The copy constructor copies a zoo with all its animals.
23     /// @param[in] z is the zoo to copy.
24     ///
25     /// The CopyCTor is implemented only when it becomes necessary in use.
26     Zoo(Zoo const& z);
27
28     /// @brief All animals must be released.
29     ///
30     /// Dynamic memory was requested.
31     ~Zoo();
32
33     /// @brief Assign operator, implemented with elegant swap function.
34     /// @param[in] z is the assigned zoo.
35     ///
36     /// The zoo is copied when called and then the containers are swapped.
37     /// When exiting the method, the swapped zoo is deleted!
38     void operator = (Zoo z);
39
40     /// @brief Adds an animal to the zoo.
41     /// @param[in] ani is a constant pointer to an animal.
42     void Add(Animal const* ani);
43
44     /// @brief Returns a string representation of a zoo and all its animals.
45     /// @returns a string of a zoo with all attributes.
46     virtual std::string ToString() const override;
```

```
47
48  /// Constant Iterator to Animals.
49  using CItor = std::vector<Animal const*>::const_iterator;
50
51  /// @brief Access to begin of the the zoo via iterators.
52  /// @returns a constant iterator to the first animal.
53  CItor cbegin() const;
54
55  /// @brief Access to the end in the zoo via iterators.
56  /// @returns a constant iterator behind the last animal.
57  CItor cend() const;
58
59 private:
60  /// @brief Polymorphic container of animals.
61  /// @details The container knows only the common interface of all animals.
62  std::vector<Animal const*> mAnimals;
63 };
64
65
66
67 #endif //ZOO_H
```

6.10 Zoo.cpp

```
1 #include <algorithm>
2 #include <sstream>
3 #include "Zoo.h"
4 using namespace std;
5
6 Zoo::Zoo(Zoo const& z)
7 {
8     for_each(z.cbegin(), z.cend(), [&](auto ani) {Add(ani->Clone()); });
9 }
10
11 Zoo::~Zoo()
12 {
13     for (auto ani : mAnimals)
14     {
15         delete ani;
16     }
17     mAnimals.clear();
```

```
18 }
19
20 void Zoo::operator=(Zoo z)
21 {
22     swap(mAnimals, z.mAnimals);
23 }
24
25 void Zoo::Add(Animal const* ani)
26 {
27     if (ani == nullptr)
28         throw invalid_argument{ "null_pointer_param_in_Zoo::Add(...)" };
29     mAnimals.push_back(ani);
30 }
31
32 std::string Zoo::ToString() const
33 {
34     ostringstream strStream;
35     strStream << "This_zoo_has_" << mAnimals.size()
36             << "_animals:" << endl;
37     for (auto const ani : mAnimals)
38     {
39         strStream << ani->ToString();
40     }
41     return strStream.str();
42 }
43
44 Zoo::CIter Zoo::cbegin() const
45 {
46     return mAnimals.cbegin();
47 }
48
49 Zoo::CIter Zoo::cend() const
50 {
51     return mAnimals.cend();
52 }
```

6.11 Testtreiber - SimpleAnimal.cpp

```
1 /// @mainpage Example for a doxygen documentation
2 /// The implementation of SimpleAnimal models different
3 /// animal species that are stored in a zoo. In the test driver,
```

```
4  /// individual animals are first created and their methods tested.
5  /// Then animals are added to the zoo and the zoo's methods
6  /// are tested as well.
7  ///
8  /// @author Franz Wiesinger
9  /// @date 10.06.2022
10 /// @version 1.0
11
12 #include <iostream>
13 #include <vector>
14 #include <algorithm>
15 #include <sstream>
16 #include <vld.h>
17 #include "Cat.h"
18 #include "Dog.h"
19 #include "Zoo.h"
20
21 using namespace std;
22
23 /// @brief OK is a constant for a positive test result.
24 static short const OK = 0;
25 /// @brief NOK is a constant for a negative test result.
26 static short const NOK = 1;
27
28
29 /// @brief Test function for verifying the weight of an Animal.
30 ///
31 /// Compares the actual weight of the animal to the expected weight
32 /// and outputs the test result.
33 ///
34 /// @param[in] animal The Animal object whose weight will be tested.
35 /// @param[in] expectedWeight The expected weight of the animal.
36 void TestAnimalWeight(const Animal& animal, size_t expectedWeight) {
37     size_t actualWeight = animal.GetWeight();
38     if (actualWeight == expectedWeight) {
39         std::cout << "[PASS]_Animal_weight_test_passed." << std::endl;
40     }
41     else {
42         std::cout << "[FAIL]_Expected_weight:_ " << expectedWeight
43             << ",_but_got:_ " << actualWeight << std::endl;
44     }
45 }
46
47 /// @brief Test function for verifying the ID of an Animal.
```

```
48 ///
49 /// Compares the actual ID of the animal to the expected ID
50 /// and outputs the test result.
51 ///
52 /// @param[in] animal The Animal object whose ID will be tested.
53 /// @param[in] expectedId The expected unique ID of the animal.
54 void TestAnimalId(const Animal& animal, size_t expectedId) {
55     size_t actualId = animal.GetId();
56     if (actualId == expectedId) {
57         std::cout << "[PASS]_Animal_ID_test_passed." << std::endl;
58     }
59     else {
60         std::cout << "[FAIL]_Expected_ID:_ " << expectedId
61             << ",_but_got:_ " << actualId << std::endl;
62     }
63 }
64
65 /// @brief Test function for verifying the sound produced by an Animal.
66 ///
67 /// Calls the 'GiveTongue' method of the animal and compares the output
68 /// with the expected sound.
69 ///
70 /// @param[in] animal The Animal object whose sound will be tested.
71 /// @param[in] expectedSound The expected sound as a string.
72 void TestAnimalSound(const Animal& animal, const std::string& expectedSound) {
73     //Switching the std::cout buffer to a stringstream:
74     //all output written to std::cout will be redirected to the string stream os i
75     std::ostringstream os;
76     std::streambuf* originalCoutBuffer = std::cout.rdbuf();
77     std::cout.rdbuf(os.rdbuf());
78
79     animal.GiveTongue();
80
81     //reset the std::cout buffer:
82     //The original buffer of std::cout is restored so that future outputs
83     //appear on the console again.
84     std::cout.rdbuf(originalCoutBuffer);
85     std::string actualSound = os.str();
86     if (actualSound == expectedSound) {
87         std::cout << "[PASS]_Animal_sound_test_passed." << std::endl;
88     }
89     else {
90         std::cout << "[FAIL]_Expected:_ " << expectedSound
91             << ",_but_got:_ " << actualSound << std::endl;
```

```
92     }
93 }
94
95 /// @brief Test function for verifying the string representation of an Animal.
96 ///
97 /// Calls the 'ToString' method of the animal and compares the returned string
98 /// with the expected string.
99 ///
100 /// @param[in] animal The Animal object whose string representation will be tested
101 /// @param[in] expectedString The expected string output from the animal.
102 void TestAnimalToString(const Animal& animal, const std::string& expectedString)
103 {
104     std::string actualString = animal.ToString();
105     if (actualString == expectedString) {
106         std::cout << "[PASS]_Animal_ToString_test_passed." << std::endl;
107     }
108     else {
109         std::cout << "[FAIL]_Expected:_ " << expectedString << ",_but_got:_ "
110             << actualString << std::endl;
111     }
112 }
113
114 /// @brief Test function for verifying the cloning behavior of an Animal.
115 ///
116 /// Creates a clone of the animal using the 'Clone' method and compares
117 /// its weight and ID to the original animal.
118 ///
119 /// @param[in] animal The Animal object to be cloned.
120 void TestAnimalClone(const Animal& animal) {
121     const Animal* clonedAnimal = animal.Clone();
122     if (clonedAnimal->GetWeight() == animal.GetWeight() &&
123         clonedAnimal->GetId() == animal.GetId()) {
124         std::cout << "[PASS]_Animal_cloning_test_passed." << std::endl;
125     }
126     else {
127         std::cout << "[FAIL]_Animal_cloning_test_failed." << std::endl;
128     }
129     delete clonedAnimal; // Make sure to free memory from cloning
130 }
131
132 /// @brief Test function for verifying the contents of a Zoo.
133 ///
134 /// Compares the string output of the zoo's 'ToString' method
135 /// with the expected zoo string.
136 ///
```

```
136 /// @param[in] zoo The Zoo object to be tested.
137 /// @param[in] expectedZooString The expected string representation of the zoo.
138 void TestZooContents(const Zoo& zoo, const std::string& expectedZooString) {
139     std::string actualZooString = zoo.ToString();
140     if (actualZooString == expectedZooString) {
141         std::cout << "[PASS]_Zoo_ToString_test_passed." << std::endl;
142     }
143     else {
144         std::cout << "[FAIL]_Expected_zoo_string:\n" << expectedZooString
145             << "\nBut_got:\n" << actualZooString << std::endl;
146     }
147 }
148
149 /// @brief Test function for handling exceptions during invalid animal creation.
150 ///
151 /// Tests if the appropriate exception is thrown when attempting to create
152 /// an animal with an invalid weight.
153 ///
154 /// @return void This function does not return a value.
155 void TestInvalidAnimalCreation() {
156     try {
157         Dog invalidDog(0); // Invalid weight, should throw exception
158         std::cout << "[FAIL]_Expected_exception_for_invalid_dog_weight_not_thrown."
159             << std::endl;
160     }
161     catch (const std::invalid_argument& e) {
162         std::cout << "[PASS]_Exception_caught:_" << e.what() << std::endl;
163     }
164
165     try {
166         Cat invalidCat(-5); // Invalid weight, should throw exception
167         std::cout << "[FAIL]_Expected_exception_for_invalid_cat_weight_not_thrown."
168             << std::endl;
169     }
170     catch (const std::invalid_argument& e) {
171         std::cout << "[PASS]_Exception_caught:_" << e.what() << std::endl;
172     }
173 }
174
175 /// @brief Test function for handling null animal addition to the Zoo.
176 ///
177 /// Tests if the appropriate exception is thrown when attempting to add
178 /// a null pointer as an animal to the zoo.
179 ///
```

```
180 /// @return void This function does not return a value.
181 void TestZooNullAnimal() {
182     Zoo zoo;
183     try {
184         zoo.Add(nullptr); // Trying to add null animal, should throw exception
185         std::cout << "[FAIL]_Expected_exception_for_null_animal_not_thrown."
186                     << std::endl;
187     }
188     catch (const std::invalid_argument& e) {
189         std::cout << "[PASS]_Exception_caught:_ " << e.what() << std::endl;
190     }
191 }
192
193
194 /// Start of the testdriver.
195 int main()
196 {
197     try
198     {
199         // Additional Exception Handling Tests
200         std::cout << "Testing_Invalid_Animal_Creation..." << std::endl;
201         TestInvalidAnimalCreation();
202
203         std::cout << "Testing_Null_Animal_Addition_to_Zoo..." << std::endl;
204         TestZooNullAnimal();
205
206         // Create animals
207         Dog dog1(25); // A dog with weight 25
208         Dog dog2(30); // Another dog with weight 30
209         Cat cat1(10); // A cat with weight 10
210
211         // Test Animal Weight
212         std::cout << "Testing_Animal_Weights..." << std::endl;
213         TestAnimalWeight(dog1, 25);
214         TestAnimalWeight(dog2, 30);
215         TestAnimalWeight(cat1, 10);
216
217         // Test Animal ID
218         std::cout << "Testing_Animal_IDs..." << std::endl;
219         TestAnimalId(dog1, 2); // Assuming first created dog has ID 0
220         TestAnimalId(dog2, 3); // Assuming second created dog has ID 1
221         TestAnimalId(cat1, 4); // Assuming first created cat has ID 2
222
223         // Test Animal Sounds
```

```
224     std::cout << "Testing_Animal_Sounds..." << std::endl;
225     TestAnimalSound(dog1, "bark\n");
226     TestAnimalSound(dog2, "bark\n");
227     TestAnimalSound(cat1, "miaow\n");
228
229     // Test Animal ToString
230     std::cout << "Testing_Animal_ToString..." << std::endl;
231     TestAnimalToString(dog1, "ID:2_I'm_a_dog_and_my_weight_is_25\n");
232     TestAnimalToString(dog2, "ID:3_I'm_a_dog_and_my_weight_is_30\n");
233     TestAnimalToString(cat1, "ID:4_I'm_a_cat_and_my_weight_is_10\n");
234
235     // Test Animal Cloning
236     std::cout << "Testing_Animal_Cloning..." << std::endl;
237     TestAnimalClone(dog1);
238     TestAnimalClone(cat1);
239
240     // Create a Zoo and add animals
241     Zoo zoo;
242     zoo.Add(new Dog{ dog1 });
243     zoo.Add(new Dog{ dog2 });
244     zoo.Add(new Cat{ cat1 });
245
246     // Expected Zoo string output
247     std::string expectedZooString = "This_zoo_has_3_animals:\n"
248         "ID:2_I'm_a_dog_and_my_weight_is_25\n"
249         "ID:3_I'm_a_dog_and_my_weight_is_30\n"
250         "ID:4_I'm_a_cat_and_my_weight_is_10\n";
251
252
253     // Test Zoo Contents
254     std::cout << "Testing_Zoo_Contents..." << std::endl;
255     TestZooContents(zoo, expectedZooString);
256 }
257 catch (invalid_argument const& ex)
258 {
259     cerr << ex.what() << endl;
260     return 1;
261 }
262 catch (bad_alloc const& ex)
263 {
264     cerr << ex.what() << endl;
265     return 1;
266 }
267 catch (...)
```

```
268     {  
269         cerr << "unhandled_exception" << endl;  
270         return 1;  
271     }  
272 }
```